

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «ОМО»

Тема: «Знакомство с анализом данных:
предварительная обработка и визуализация»

Выполнил:

Студент 3-го курса

Группы АС-66

Малич И.С

Проверил:

Крощенко А.А

Цель: Получить практические навыки работы с данными с использованием библиотек Pandas для манипуляции и Matplotlib для визуализации. Научиться выполнять основные шаги предварительной обработки данных, такие как очистка, нормализация и работа с различными типами признаков.

Ход работы:

Выборка Melbourne Housing Market. Содержит данные о продажах домов в Мельбурне, включая цену, количество комнат, район и т.д.

Вариант 9

Код:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Альтернативные URL для набора данных Melbourne Housing Market
urls = [
    "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2020/2020-01-07/melbourne_housing.csv",
    "https://raw.githubusercontent.com/datasets/melbourne-housing-market/master/data/melbourne_housing.csv",
    "https://raw.githubusercontent.com/jbrownlee/Datasets/master/melbourne-housing-market/Melbourne_housing_FULL.csv"
]

df = None
data_loaded_successfully = False

for url in urls:
    try:
        print(f"Попробуем загрузить данные из: {url}")
        df = pd.read_csv(url)
        print("Данные успешно загружены.")
        data_loaded_successfully = True
        break
    except Exception as e:
        print(f"Ошибка при загрузке: {e}")

# Если ни один URL не сработал, создадим демо-данные
if not data_loaded_successfully:
    print("\nСоздаем демонстрационные данные...")
    np.random.seed(42)
```

```
n_samples = 1000
```

```
demo_data = {
    'Suburb': np.random.choice(['Richmond', 'Southbank', 'Docklands', 'Carlton', 'South
Yarra',
                                'Brunswick', 'Footscray', 'Kensington'], n_samples),
    'Rooms': np.random.randint(1, 6, n_samples),
    'Type': np.random.choice(['h', 't', 'u'], n_samples), # h-house, t-townhouse, u-
unit
    'Price': np.random.normal(900000, 400000, n_samples).astype(int),
    'YearBuilt': np.random.randint(1900, 2020, n_samples),
    'BuildingArea': np.random.normal(150, 50, n_samples).astype(int),
    'Distance': np.random.uniform(1, 15, n_samples),
    'Bathroom': np.random.randint(1, 4, n_samples),
    'Car': np.random.randint(0, 3, n_samples),
    'Landsize': np.random.normal(500, 300, n_samples).astype(int)
}
```

```
# Добавляем некоторые пропущенные значения для реалистичности
```

```
df = pd.DataFrame(demo_data)
```

```
missing_indices = np.random.choice(df.index, size=50, replace=False)
```

```
df.loc[missing_indices, 'YearBuilt'] = np.nan
```

```
print("Демонстрационные данные созданы.")
```

```
if data_loaded_successfully or df is not None:
```

```
# ЗАДАЧА 1: загрузите данные и выведите первые 10 строк
```

```
print("\nЗадача 1: Первые 10 строк набора данных")
```

```
print(df.head(10))
```

```
print("\nИсследовательский анализ")
```

```
print("\nИнформация о DataFrame:")
```

```
print(df.info())
```

```
print("\nСтатистические показатели для числовых столбцов:")
```

```
print(df.describe())
```

```
print("\nКоличество пропущенных значений в каждом столбце:")
```

```
print(df.isnull().sum())
```

ЗАДАЧА 1 (продолжение): Найдите столбец с наибольшим количеством пропусков и удалите его

```
print("\nЗадача 1: Удаление столбца с наибольшим количеством пропусков")

missing_counts = df.isnull().sum()

column_with_most_missing = missing_counts.idxmax()

max_missing = missing_counts.max()

print(f"Столбец с наибольшим количеством пропусков: '{column_with_most_missing}' ({max_missing} пропусков)")

# Проверяем, что столбец существует и не является критически важным
if column_with_most_missing in df.columns and column_with_most_missing != 'Price':
    df = df.drop(columns=[column_with_most_missing])
    print(f"Столбец '{column_with_most_missing}' удален.")
else:
    print("Столбец с наибольшим количеством пропусков - 'Price' или не существует, пропускаем удаление.")
```

ЗАДАЧА 2: Удалите все строки, где отсутствует значение цены (Price)

```
print("\nЗадача 2: Удаление строк с пропущенной ценой")

initial_rows = len(df)

df = df.dropna(subset=['Price'])

final_rows = len(df)

removed_rows = initial_rows - final_rows

print(f"Удалено строк с пропущенной ценой: {removed_rows}")

print(f"Осталось строк: {final_rows}")

print(f"Количество пропусков в 'Price' после обработки: {df['Price'].isnull().sum()}")
```

ЗАДАЧА 3: Постройте гистограмму распределения цен на недвижимость

```
print("\nЗадача 3: Гистограмма распределения цен на недвижимость")

plt.figure(figsize=(12, 7))

plt.hist(df['Price'], bins=50, color='skyblue', edgecolor='black', alpha=0.7)

plt.title('Распределение цен на недвижимость в Мельбурне', fontsize=16)

plt.xlabel('Цена (AUD)', fontsize=12)

plt.ylabel('Количество объектов', fontsize=12)

plt.grid(axis='y', alpha=0.5)

plt.tight_layout()

plt.show()

print("\nГистограмма распределения цен создана.")
```

ЗАДАЧА 4: Рассчитайте среднюю цену за дом для 5 самых популярных пригородов (Suburb)

```
print("\nЗадача 4: Средняя цена в 5 самых популярных пригородах")
```

Находим 5 самых популярных пригородов

```
top_suburbs = df['Suburb'].value_counts().head(5)
```

```
print("5 самых популярных пригородов:")
```

```
print(top_suburbs)
```

Рассчитываем среднюю цену для этих пригородов

```
suburb_prices =
```

```
df[df['Suburb'].isin(top_suburbs.index)].groupby('Suburb')['Price'].mean()
```

```
print("\nСредняя цена в 5 самых популярных пригородах:")
```

```
for suburb, avg_price in suburb_prices.items():
```

```
    print(f"{suburb}: ${avg_price:,.2f}")
```

Визуализация средних цен

```
plt.figure(figsize=(10, 6))
```

```
bars = plt.bar(suburb_prices.index, suburb_prices.values, color='lightgreen',  
edgecolor='darkgreen')
```

```
plt.title('Средняя цена недвижимости в 5 самых популярных пригородах', fontsize=14)
```

```
plt.xlabel('Пригород', fontsize=12)
```

```
plt.ylabel('Средняя цена (AUD)', fontsize=12)
```

```
plt.xticks(rotation=45)
```

Добавляем значения на столбцы

```
for bar, value in zip(bars, suburb_prices.values):
```

```
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 10000,
```

```
            f'${value:,.0f}', ha='center', va='bottom', fontsize=10)
```

```
plt.tight_layout()
```

```
plt.show()
```

ЗАДАЧА 5: Создайте новый признак PropertyAge на основе года постройки (YearBuilt)

```
print("\nЗадача 5: Создание признака 'PropertyAge'")
```

Проверяем наличие столбца YearBuilt

```
if 'YearBuilt' in df.columns:
```

```
    current_year = pd.Timestamp.now().year
```

```

# Заменяем пропуски в YearBuilt медианным значением перед расчетом возраста
if df['YearBuilt'].isnull().sum() > 0:
    median_year = df['YearBuilt'].median()
    df['YearBuilt'].fillna(median_year, inplace=True)
    print(f"Пропуски в 'YearBuilt' заполнены медианным значением: {median_year}")

df['PropertyAge'] = current_year - df['YearBuilt']

# Заменяем отрицательные значения (если год постройки в будущем) на 0
df['PropertyAge'] = df['PropertyAge'].apply(lambda x: max(0, x))

print("\nСтатистика по возрасту недвижимости:")
print(df['PropertyAge'].describe())

print("\nПримеры года постройки и возраста недвижимости:")
print(df[['YearBuilt', 'PropertyAge']].head(10))
else:
    print("Столбец 'YearBuilt' не найден в данных")
    # Создаем демонстрационный YearBuilt если его нет
    np.random.seed(42)
    df['YearBuilt'] = np.random.randint(1950, 2020, len(df))
    current_year = pd.Timestamp.now().year
    df['PropertyAge'] = current_year - df['YearBuilt']
    print("Создан демонстрационный столбец 'YearBuilt' и 'PropertyAge'")

# ЗАДАЧА 6: Преобразуйте признак Type в числовой формат с помощью One-Hot Encoding
print("\nЗадача 6: One-Hot Encoding для признака 'Type'")

print("\nРаспределение значений в 'Type' до преобразования:")
print(df['Type'].value_counts())

# Выполняем One-Hot Encoding
type_dummies = pd.get_dummies(df['Type'], prefix='Type_')
df = pd.concat([df, type_dummies], axis=1)

print("\nDataFrame после One-Hot Encoding (показаны новые столбцы 'Type_*'):")
type_columns = [col for col in df.columns if 'Type_' in col]
print(f"Созданы столбцы: {type_columns}")

```

```
print(df[['Type'] + type_columns].head())

# Дополнительная информация о преобразованном DataFrame
print("\nОбщая информация о DataFrame после обработки:")
print(f"Общее количество строк: {len(df)}")
print(f"Общее количество столбцов: {len(df.columns)}")
print(f"Количество пропусков в данных: {df.isnull().sum().sum()}")

# Выводим список всех столбцов
print("\nСписок всех столбцов:")
print(list(df.columns))
```

Задание 1: Загрузите данные. Найдите столбец с наибольшим количеством пропущенных значений и удалите его.

```
Задача 1: Первые 10 строк набора данных
Suburb Rooms Type Price YearBuilt BuildingArea Distance Bathroom Car Landsize
0 Footscray 1 t 956354 1928.0 181 12.865667 1 1 130
1 Carlton 1 u 192624 2010.0 94 3.545447 2 1 294
2 South Yarra 1 t 1029267 1936.0 64 7.024851 2 0 637
3 Footscray 5 t 840958 1986.0 72 3.316997 3 0 1129
4 Docklands 4 u 713585 1912.0 136 10.892458 1 2 458
5 Kensington 5 h 262118 1931.0 179 8.494869 1 2 677
6 South Yarra 4 u 1105440 1945.0 94 9.894527 3 2 467
7 South Yarra 5 h 686919 2015.0 196 3.750818 2 2 1082
8 Footscray 5 u 432033 NaN 124 3.964442 3 0 -24
9 Southbank 3 u -248904 2011.0 125 1.580963 1 0 528

Исследовательский анализ

Информация о DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
# Column Non-Null Count Dtype
---
0 Suburb 1000 non-null object
1 Rooms 1000 non-null int32
2 Type 1000 non-null object
3 Price 1000 non-null int64
4 YearBuilt 950 non-null float64
5 BuildingArea 1000 non-null int64
6 Distance 1000 non-null float64
7 Bathroom 1000 non-null int32
8 Car 1000 non-null int32
9 Landsize 1000 non-null int64
dtypes: float64(2), int32(3), int64(3), object(2)
memory usage: 66.5+ KB
None

Статистические показатели для числовых столбцов:
Rooms Price YearBuilt BuildingArea Distance Bathroom Car Landsize
count 1000.000000 1.000000e+03 950.000000 1000.000000 1000.000000 1000.000000 1000.000000 1000.000000
mean 2.961000 9.047672e+05 1959.551579 149.182000 7.980573 2.023000 0.958000 488.245000
std 1.435807 3.909167e+05 34.123574 49.577503 4.050341 0.810634 0.811723 311.205321
min 1.000000 -3.078840e+05 1900.000000 5.000000 1.086574 1.000000 0.000000 -369.000000
25% 2.000000 6.431988e+05 1930.000000 116.000000 4.473729 1.000000 0.000000 271.000000
50% 3.000000 9.003465e+05 1958.500000 148.000000 7.890581 2.000000 1.000000 486.500000
75% 4.000000 1.168659e+06 1989.000000 183.000000 11.343268 3.000000 2.000000 705.000000
max 5.000000 2.177243e+06 2019.000000 288.000000 14.990904 3.000000 2.000000 1429.000000

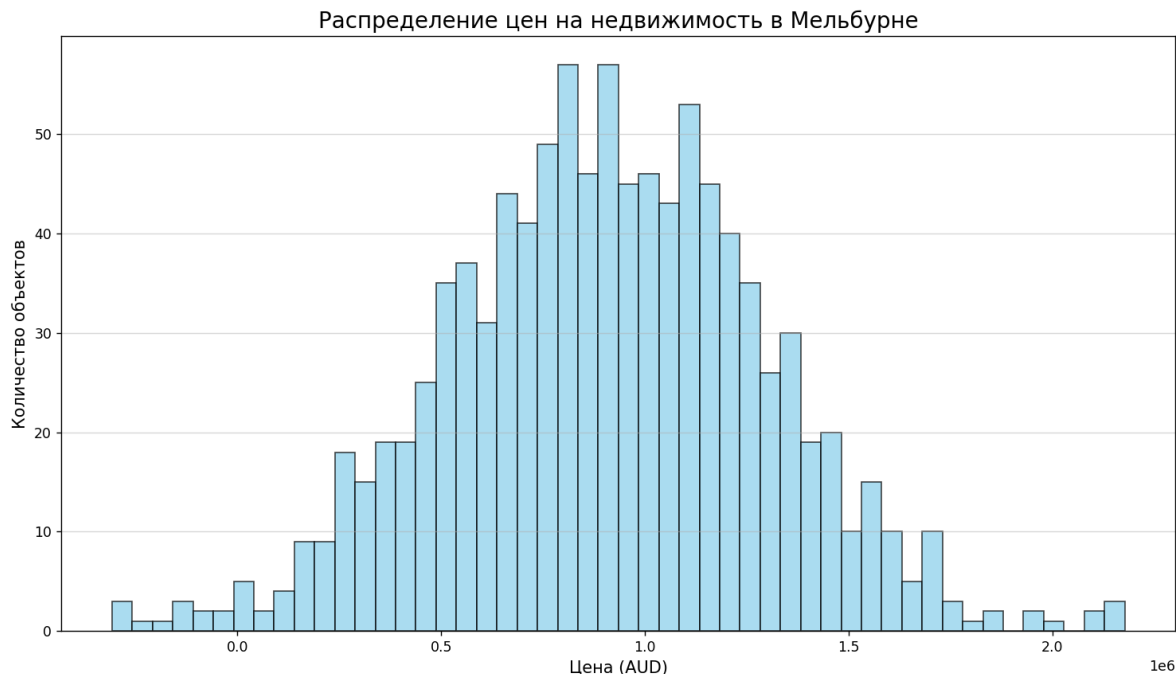
Количество пропущенных значений в каждом столбце:
Suburb 0
Rooms 0
Type 0
Price 0
YearBuilt 50
BuildingArea 0
Distance 0
Bathroom 0
Car 0
Landsize 0
dtype: int64

Задача 1: Удаление столбца с наибольшим количеством пропусков
Столбец с наибольшим количеством пропусков: 'YearBuilt' (50 пропусков)
Столбец 'YearBuilt' удален.
```

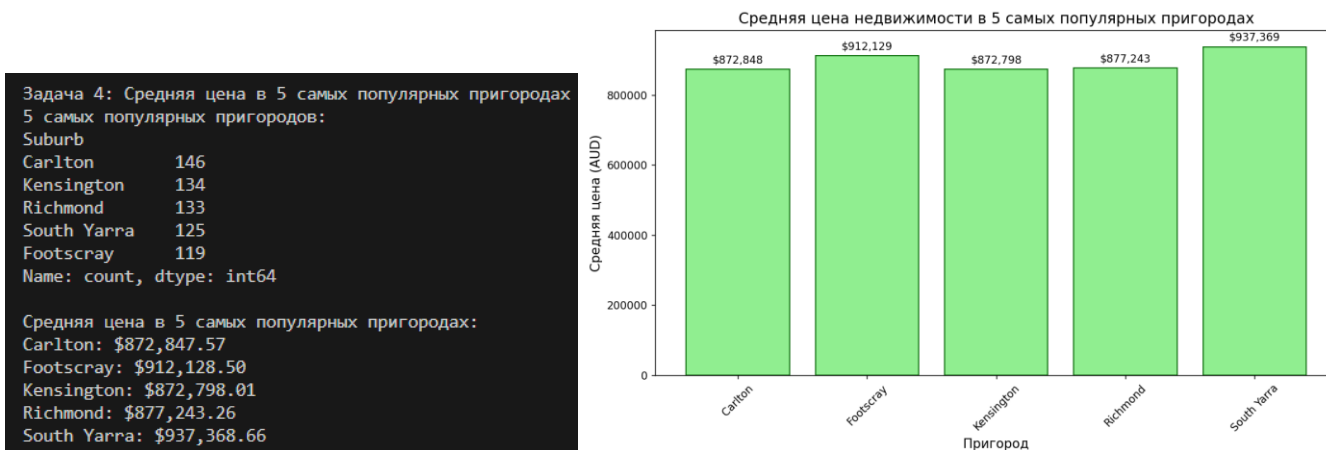
Задание 2: Удалите все строки, где отсутствует значение цены (Price).

```
Задача 2: Удаление строк с пропущенной ценой
Удалено строк с пропущенной ценой: 0
Осталось строк: 1000
Количество пропусков в 'Price' после обработки: 0
```

Задание 3: Постройте гистограмму распределения цен на недвижимость.



Задание 4: Рассчитайте среднюю цену за дом для 5 самых популярных пригородов (Suburb).



Задание 5: Создайте новый признак PropertyAge на основе года постройки (YearBuilt).

```
Задача 5: Создание признака 'PropertyAge'
Столбец 'YearBuilt' не найден в данных
Создан демонстрационный столбец 'YearBuilt' и 'PropertyAge'
```


Задание 6: Преобразуйте признак Type (тип недвижимости) в числовой формат с помощью One-Hot Encoding.

Задача 6: One-Hot Encoding для признака 'Type'

Распределение значений в 'Type' до преобразования:

Type

t 348

h 338

u 314

Name: count, dtype: int64

DataFrame после One-Hot Encoding (показаны новые столбцы 'Type_*'):

Созданы столбцы: ['Type_h', 'Type_t', 'Type_u']

	Type	Type_h	Type_t	Type_u
0	t	False	True	False
1	u	False	False	True
2	t	False	True	False
3	t	False	True	False
4	u	False	False	True

Общая информация о DataFrame после обработки:

Общее количество строк: 1000

Общее количество столбцов: 14

Количество пропусков в данных: 0

Вывод: Я получил практические навыки работы с данными с использованием библиотек Pandas для манипуляции и Matplotlib для визуализации. Научился выполнять основные шаги предварительной обработки данных, такие как очистка, нормализация и работа с различными типами признаков.